

Q1 Handling ArithmeticException

10 marks

Write a Java program to divide two integers entered by the user. If the denominator is zero, handle the exception using a try-catch block and display an appropriate error message.

Your Code

```
1 import java.util.*;
2 public class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int a=sc.nextInt();
6         int b=sc.nextInt();
7         try{
8             System.out.println(a/b);
9         }
10        catch(ArithmeticException e){
11            System.out.println("Cannot divide by zero");
12        }
13        sc.close();
14    }
15 }
```

Input

20
0

Output

Cannot divide by zero



Run



Save

Q2 Handling ArrayIndexOutOfBoundsException

10 marks

Write a Java program to create an integer array of five elements. Ask the user to enter an index and display the corresponding element. If the index is outside the valid range, handle the exception and display an error message.

Your Code

```
1 import java.util.*;
2 public class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int n[]={1,6,4,2,9};
6         System.out.println("Give a index value");
7         int i=sc.nextInt();
8         try{
9             System.out.println("Element is: "+n[i]);
10        }
11        catch(ArrayIndexOutOfBoundsException e){
12            System.out.println("The index is out of range");
13        }
14        sc.close();
15    }
16 }
17 }
```

Input

10

Output

```
Give a index value
The index is out of range
```



Run



Save

Q3 Handling NumberFormatException

10 marks

Write a Java program to accept a number as a string and convert it into an integer using `Integer.parseInt()`. If the input is not a valid integer, handle the exception and display an appropriate message.

Your Code

```
1 import java.util.*;
2 public class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String s=sc.nextLine();
6         try{
7             System.out.println("Integer is:"+Integer.parseInt(s));
8         }
9         catch (NumberFormatException e){
10             System.out.println("String cannot be a number");
11         }
12         sc.close();
13
14     }
15 }
```

Input

abc

Output

String cannot be a number



Run



Save

Q4 Handling Multiple Exceptions

10 marks

Write a Java program that performs the following operations:

Accept two integers from the user.

Divide the first number by the second.

Store the result in an array.

Handle both `ArithmeticException` and `ArrayIndexOutOfBoundsException` using multiple catch blocks.

Your Code

```
1 import java.util.*;
2 public class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int a=sc.nextInt();
6         int b=sc.nextInt();
7         int arr[]=new int[5];
8         try{
9             System.out.println(a/b);
10            int index=sc.nextInt();
11            arr[index]=a/b;
12            System.out.println("stored successfully: "+arr[index]);
13        }
14        catch(ArithmeticException e){
15            System.out.println("Denominator Cannot be 0");
16        }
17        catch(ArrayIndexOutOfBoundsException e){
18            System.out.println("Index is out of range");
19        }
20        sc.close();
21    }
22 }
```

Input

36 0
8

Output

Denominator Cannot be 0



Run



Save